CompletableFuture

CompletableFuture

- **Asynchrone Berechnungen sind schwer nachvollziehbar** – Ablauf nicht linear, Aktionen (Callbacks) oft verteilt oder verschachtelt.
- **Fehlerhandling erschwert die Komplexität** – Fehler können an beliebigen Stellen im Ablauf auftreten.
- **Java 5: Future als erster Ansatz** – liefert ein Ergebnis, bietet aber **keine Möglichkeit**, Berechnungen zu verketten oder Fehler elegant zu behandeln.

CompletableFuture

- **Java 8: Einführung von CompletableFuture** – implementiert zusätzlich CompletionStage.
- **CompletionStage** beschreibt einen **kombinierbaren asynchronen Berechnungsschritt**.
- **CompletableFuture als Baustein & Framework** – rund **50 Methoden** zum Verketten, Kombinieren, Ausführen und **Fehlerbehandeln** asynchroner Schritte.

CompletableFuture

- **CompletableFuture implementiert Future**, erweitert es aber um aktive **Vervollständigungslogik**.
- Eine Instanz kann **leer erzeugt** und an Konsumenten verteilt werden; später wird sie über **complete(...)** abgeschlossen.
- Konsumenten können per **get()** blockierend auf das Ergebnis warten.
- Typisches Muster:
    - Methode erstellt ein **CompletableFuture**,
    - startet eine Berechnung in einem **anderen Thread**,
    - gibt das Future **sofort** zurück,
    - und ruft nach Abschluss der Berechnung **complete(...)** auf.

Hier legen wir das CompletableFuture an. Es ist noch nicht abgeschlossen.

```
    <String> calculateAsync()
                                throws InterruptedException {
        CompletableFuture<String> completableFuture =
            new CompletableFuture<>();

        Executors.newCachedThreadPool().submit(() -> {
            Thread.sleep(500);
            completableFuture.complete("Hello");
            return null;
        });

        return completableFuture;
    }
```

Innerhalb des Executors wird das Future dann abgeschlossen.

CompletableFuture aus Wert

```
Future<String> completableFuture =
  CompletableFuture.completedFuture("Hello");

// ...

String result = completableFuture.get();
assertEquals("Hello", result);
```

Wir können auch direkt ein abschlossenes Future anlegen, wenn wir das Ergebnis bereits wissen.

CompletableFuture aus Supplier/Runnable

```
CompletableFuture<String> future
  = CompletableFuture.supplyAsync(() -> "Hello");

// ...

assertEquals("Hello", future.get());
```

Ein CompletableFuture lässt sich auch aus einem **Runnable** oder einem **Supplier** erstellen.

Weiterverarbeitung von Ergebnissen

```
CompletableFuture<String> completableFuture
  = CompletableFuture.supplyAsync(() -> "Hello");

CompletableFuture<String> future = completableFuture
  .thenApply(s -> s + " World");

assertEquals("Hello World", future.get());
```

thenApply
Bei CompletableFuture werden Lambdas verwendet, um das Ergebnis weiterzuverarbeiten.

Weiterverarbeitung von Ergebnissen

enAccept
llen wir einen Java 8-
nsumer verwenden, so
wenden wir diese
thode.

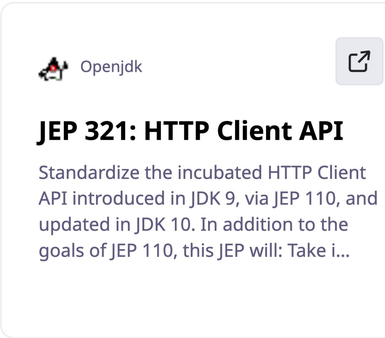
```
CompletableFuture<String> completableFuture
  = CompletableFuture.supplyAsync(() -> "Hello");

CompletableFuture<Void> future1 = completableFuture
  .thenAccept(s -> System.out.println("Computation returned
      : " + s)); // Für Consumer

CompletableFuture<Void> future2 = completableFuture
  .thenRun(() -> System.out.println("Computation finished."
      )); // FÜr Runnable

future.get();
```

thenRun
Diese Methode erwartet ein Runnable als Parameter, welches als Lambda aber genau wie ein Consumer aussieht.

Java 11 HttpClient

Überblick

- Zeitgemäße Alternative zu HttpURLConnection
- Neue API für HTTP-Anfragen
- Verbessert Java-Webkommunikation
- Ermöglicht HTTP/2-Unterstützung
- Einfacherer und flexiblerer Code



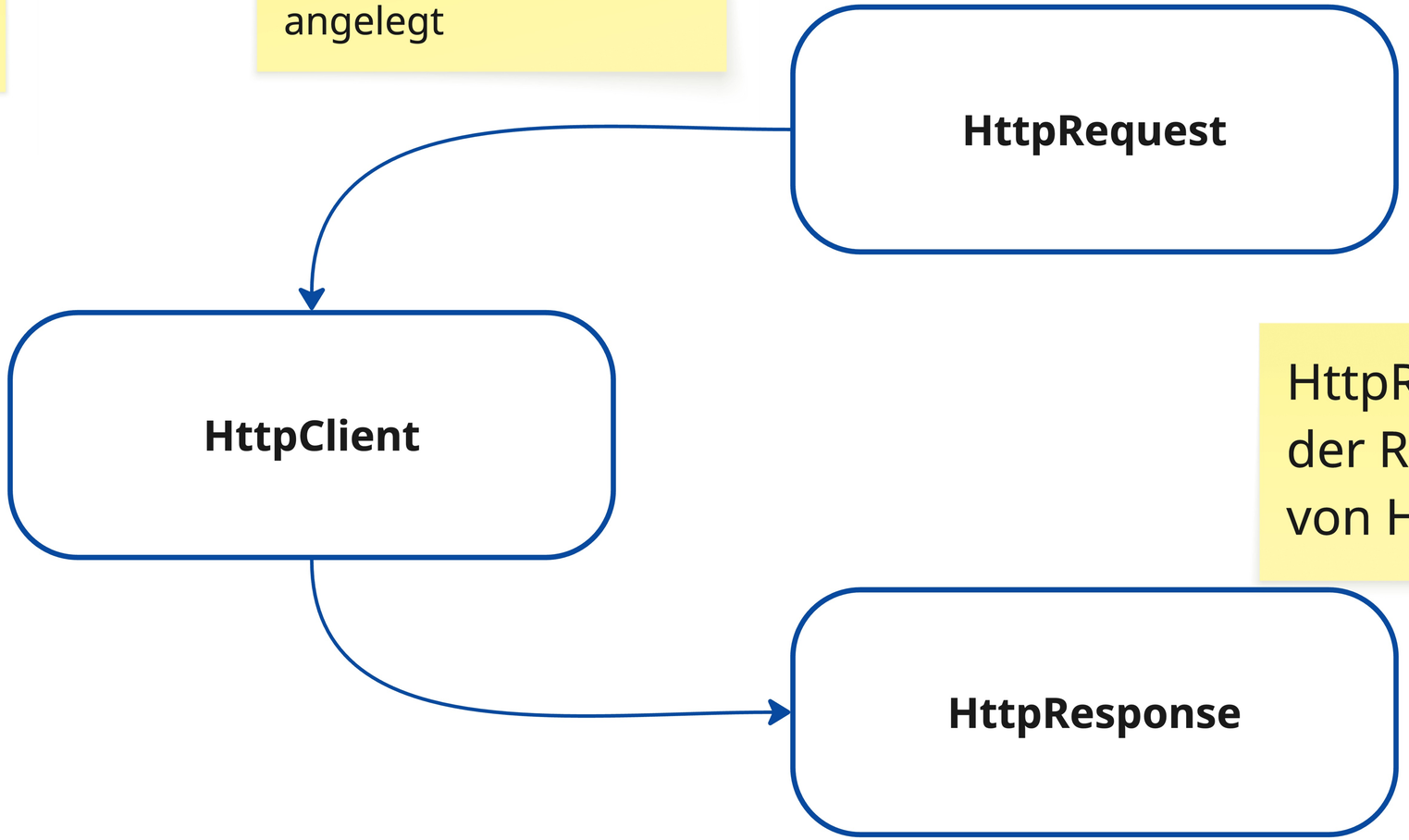
Was ist das Problem mit HttpURLConnection?

```java
// Beispiel: Eine einfache POST-Anfrage mit HttpURLConnection
// (zeigt die typischen Schwächen der alten API)

URL url = new URL("https://example.com/api");
HttpURLConnection conn = (HttpURLConnection) url.openConnection();

try {
    // Schon das Setzen der Methode ist umständlich und wirft checked Exceptions
    conn.setRequestMethod("POST");

    // Redirects, Timeouts, Header → alles manuell und inkonsistent
    conn.setInstanceFollowRedirects(false);
    conn.setConnectTimeout(5000);
    conn.setReadTimeout(5000);
    conn.setDoOutput(true);
    conn.setRequestProperty("Content-Type", "application/json");

    // Body schreiben (kein Try-with-resources möglich ohne Verrenkungen)
    OutputStream os = conn.getOutputStream();
    os.write("{\"msg\":\"hello\"}".getBytes(StandardCharsets.UTF_8));
    os.flush();
    os.close(); // manuelles Schließen zwingend

    int status = conn.getResponseCode();

    // Fehler-Stream vs. Input-Stream: umständliche if/else-Logik
    InputStream is = (status >= 400)
            ? conn.getErrorStream()
            : conn.getInputStream();

    // Stream-Auslesen ist komplett manuell
    BufferedReader br = new BufferedReader(new InputStreamReader(is));
    StringBuilder response = new StringBuilder();
    String line;
    while ((line = br.readLine()) != null) {
        response.append(line);
    }
    br.close(); // wieder manuelles Schließen

    System.out.println("Status: " + status);
    System.out.println("Response: " + response);

} finally {
    // Verbindung explizit disconnecten – vergessen viele
    conn.disconnect();
}
```

HttpClient-API

Die API besteht aus drei Hauptteilen

HttpRequest und HttpClient werden über Factories angelegt



HttpResponse ist der Rückgabetyp von HttpClient

HttpClient Builder

```
1    var client = HttpClient.newBuilder()
2                    .version(HttpClient.Version.HTTP_1_1)
3                    .followRedirects(HttpClient.Redirect.NEVER)
4                    .build();
```

Es können HTTP 1.1 und HTTP 2.0 aktiviert werden

Redirects können auf diese Weise unterbunden werden

HttpClient: Weitere Einstellungen

```
1    HttpClient.newBuilder()
2                  .connectTimeout(Duration.ofSeconds(10))
3                  .executor(Executors.newFixedThreadPool(4))
4                  .build();
```

Connect Timeout
Zeit beim Verbindungsaufbau, die vergeht, bis eine HttpConnectTimeoutException geworfen wird.

Executor
Wird verwendet, wenn asynchrone HTTP-Aufrufe vorgenommen werden.

HttpClient: Grundlegende Verwendung

```java
try(var client = HttpClient.newBuilder().build()) {
  URI uri = URI.create("https://api.restful-api.dev/objects");
  var request = HttpRequest.newBuilder().GET().uri(uri).build();
  var responseHandler = HttpResponse.BodyHandlers.ofString();

  HttpResponse<String> response = client.send(
        request,
        responseHandler
  );

  System.out.println(response.body());
}
```

HttpRequest hat einen Builder, um den Request zusammenzubauen.

Response-Bodies werden mit einem **BodyHandler** verarbeitet.

Beim Aufruf von send() erhalten wir die Antwort **synchron**.

HttpClient: BodyHandler

```java
HttpResponse<String> response = client.send(
    request,
    HttpResponse.BodyHandlers.ofString(StandardCharsets.UTF_8)
);
```

Body als **String**, optional mit **Charset**

```java
HttpResponse<byte[]> response = client.send(
    request,
    HttpResponse.BodyHandlers.ofByteArray()
);
```

Body als **Byte Array**

```java
HttpResponse<InputStream> response = client.send(
    request,
    HttpResponse.BodyHandlers.ofInputStream()
);
```

Body als **InputStream**

Vorsicht: Auf Speicher achten!

```java
Path target = Paths.get("download.bin");

HttpResponse<Path> response = client.send(
    request,
    HttpResponse.BodyHandlers.ofFile(target)
);
```

Body in **Datei** ausschreiben

HttpClient: POST

Wichtig: Content-Type Header setzen!

```
String jsonPayload = "{\"key\":\"value\"}";
HttpRequest postJsonRequest = HttpRequest.newBuilder()
  .uri(URI.create("https://api.example.com/data"))
      .header("Content-Type", "application/json")
      .POST(BodyPublishers.ofString(jsonPayload))
      .build();
HttpResponse<String> postJsonResponse =
     client.send(postJsonRequest, BodyHandlers.ofString());
```

BodyPublisher überträgt die Daten

BodyPublisher

```
HttpRequest request = HttpRequest.newBuilder(uri)
    .POST(HttpRequest.BodyPublishers.ofString("hello world"))
    .build();
```

Body aus **String**, optional mit **Charset**

```
HttpRequest request = HttpRequest.newBuilder(uri)
    .POST(HttpRequest.BodyPublishers.ofByteArray(bytes))
    .build();
```

Body aus **Byte Array**

```
HttpRequest request = HttpRequest.newBuilder(uri)
    .POST(HttpRequest.BodyPublishers.ofInputStream(() ->
        myInputStream))
    .build();
```

Body aus **InputStream**

```
HttpRequest request = HttpRequest.newBuilder(uri)
    .POST(HttpRequest.BodyPublishers.ofFile(Paths.get("data
        .json")))
    .build();
```

Body aus **Datei** auslesen

HttpClient: Asynchrone Verwendung

```
1     try(var client = HttpClient.newBuilder().build()) {
2         URI uri = URI.create("https://api.restful-api.dev/objects");
3         var request = HttpRequest.newBuilder().GET().uri(uri).build();
4         var responseHandler = HttpResponse.BodyHandlers.ofString();
5
6         CompletableFuture<HttpResponse<String>> response = client.sendAsync(
7                     request,
8                     responseHandler);
9         response.thenAccept(System.out::println);
10    }
```

Beim Aufruf von sendAsync() erhalten wir die Antwort **asynchron als CompletableFuture**.